Поиск

Современные технологии программирования

Главная

Лекции

Лекция 1: Введение в Java

Лекция 2: Основные конструкции Java

Лекция 3: Ветвление, Циклы и ООП

Лекция 4: Вложенные классы, Обобщени…

Лекция 5: Коллекции, I/O и Многопоточн…

Лекция 6: Системы сборки, JDBC и Hiber…

Практические занятия

Практика 1: Основы Java

Практика 2: Основные конструкции Java

- **Часть 1: Классы и модификаторы д…**
  - Задание 1.1: Банковский счёт
  - Задание 1.2: Модификаторы досту…
  - Задание 1.3: Ключевое слово `var`
- Часть 2: Абстрактные классы, Sealed-…
  - Задание 2.1: Иерархия сотрудников

Практическое занятие 2: Основные конструкции языка Java

Часть 1: Классы и модификаторы доступа

Задание 1.1: Банковский счёт

Напишите класс `BankAccount` с нуля. Вам даны только метод `main` и ожидаемый вывод. Вы должны самостоятельно спроектировать и реализовать весь класс.

Требования к классу `BankAccount`:

1. Приватные поля экземпляра: `owner` (String), `balance` (double), `accountNumber` (String)
2. Приватное статическое поле `totalAccounts` (int) — общий счётчик всех созданных счетов
3. Приватное статическое поле `bankName` (String) — название банка, общее для всех счетов
4. Статический блок инициализации: устанавливает `bankName = "Java Bank"` и выводит `"Банковская система инициализирована"`
5. Блок инициализации экземпляра: увеличивает `totalAccounts` и выводит `"Создание счёта #" + totalAccounts`
6. Конструктор `BankAccount(String owner, double initialBalance)` — генерирует `accountNumber` в формате `"ACC-" + totalAccounts` (например, `"ACC-1"`)
7. Метод `deposit(double amount)` — увеличивает баланс; если `amount <= 0` — выводит `"Ошибка: сумма должна быть положительной"` и не изменяет баланс
8. Метод `withdraw(double amount)` — уменьшает баланс; если средств недостаточно — выводит `"Ошибка: недостаточно средств"` и не изменяет баланс
9. Статический метод `getTotalAccounts()` — возвращает количество созданных счетов
10. Переопределённый `toString()` — возвращает строку в формате `"[ACC-1] Алиса: 1500.00 руб."`

```java
public class BankAccount {

    // Напишите весь класс самостоятельно

    public static void main(String[] args) {
        BankAccount a1 = new BankAccount("Алиса", 1000);
        BankAccount a2 = new BankAccount("Борис", 500);

        System.out.println(a1);
        System.out.println(a2);

        a1.deposit(500);
        System.out.println("После пополнения: " + a1);

        a1.withdraw(200);
        System.out.println("После снятия: " + a1);

        a1.withdraw(5000);

        a2.deposit(-100);

        System.out.println("Всего счетов: " + BankAccount.getTotalAccounts());
    }
}
```

Ожидаемый вывод:

```
Банковская система инициализирована
Создание счёта #1
Создание счёта #2
[ACC-1] Алиса: 1000.00 руб.
[ACC-2] Борис: 500.00 руб.
После пополнения: [ACC-1] Алиса: 1500.00 руб.
После снятия: [ACC-1] Алиса: 1300.00 руб.
Ошибка: недостаточно средств
Ошибка: сумма должна быть положительной
Всего счетов: 2
```

Задание 1.2: Модификаторы доступа — анализ и исправление

Дан следующий код с двумя классами, которые находятся в **разных пакетах**. Внимательно изучите код, затем ответьте на вопросы и выполните задания.

```java
package company.core;

public class Employee {
    public String name;
    protected int age;
    double salary;              // какой модификатор?
    private String password;
```

```java
    public Employee(String name, int age, double salary, String password) {
        this.name = name;
        this.age = age;
        this.salary = salary;
        this.password = password;
    }

    public String getRole() {
        return "Employee";
    }

    protected void promote(double raise) {
        this.salary += raise;
    }

    void printSummary() {
        System.out.println(name + ", " + age + " лет, " + salary + " руб.");
    }

    private boolean validatePassword(String input) {
        return password.equals(input);
    }
}
```

```java
package company.app;

import company.core.Employee;

public class HRSystem {
    public static void main(String[] args) {
        Employee emp = new Employee("Иван", 30, 80000, "secret");

        System.out.println(emp.name);            // Строка A
        System.out.println(emp.age);             // Строка B
        System.out.println(emp.salary);          // Строка C
        System.out.println(emp.password);        // Строка D
        System.out.println(emp.getRole());       // Строка E
        emp.promote(5000);                       // Строка F
        emp.printSummary();                      // Строка G
        emp.validatePassword("secret");          // Строка H
    }
}
```

Задания:

1. Для каждой строки (A–H) определите: скомпилируется ли она? Если нет — укажите причину (модификатор + пакет).
2. Заполните таблицу:

Строка	Компилируется?	Почему?
A		
B		
C		
D		
E		
F		
G		
H		

3. Создайте файл `EmployeeFixed.java`. Перепишите класс `Employee` с правильной инкапсуляцией:
    - Все поля должны быть `private`
    - Добавьте геттеры для `name`, `age`, `salary` (но **не** для `password`)
    - Метод `promote()` должен быть `public`
    - Метод `printSummary()` должен быть `public`
    - Метод `validatePassword()` остаётся `private`, но добавьте публичный метод `authenticate(String input)`, который вызывает `validatePassword()` внутри

Задание 1.3: Ключевое слово `var`

Напишите файл `VarDemo.java`, который демонстрирует возможности и ограничения `var`.

Требования:

1. Напишите 5 примеров, где `var` работает корректно (с разными типами: int, String, ArrayList, массив, ваш собственный объект). После каждого примера выведите `getClass().getSimpleName()` для проверки типа.
2. В комментариях после блока рабочего кода напишите 4 примера, где `var` **не компилируется**, с объяснением почему:
    - `var` без инициализации
    - `var` как параметр метода
    - `var` как поле класса
    - `var` с `null`

```java
public class VarDemo {
    // var field = 10; // Не компилируется — var нельзя использовать для полей класса
```

```
    public static void main(String[] args) {
        // Напишите 5 рабочих примеров с var
        // Для каждого выведите тип через getClass().getSimpleName()

        // Затем в комментариях покажите 4 случая, где var не работает
    }
}
```

Ожидаемый вывод (примерный):

```
42 -> Integer
Java -> String
[один, два] -> ArrayList
[1, 2, 3] -> int[]
BankAccount -> BankAccount
```

Часть 2: Абстрактные классы, Sealed-классы и интерфейсы

Задание 2.1: Иерархия сотрудников

Спроектируйте и реализуйте систему расчёта бонусов сотрудников. Напишите весь код с нуля. Вам дан только метод `main` и ожидаемый вывод.

Требования:

1. Абстрактный класс `Employee`:
    - Поля: `name` (protected), `baseSalary` (protected)
    - Конструктор, геттеры
    - Абстрактный метод `double calculateBonus()`
    - Обычный метод `double totalCompensation()` — возвращает `baseSalary + calculateBonus()`
    - Переопределённый `toString()` — формат: `"Имя | Оклад: X | Бонус: Y | Итого: Z"` (все суммы с `.0`)
2. Класс `Manager extends Employee`:
    - Дополнительное поле: `teamSize` (int)
    - Бонус = `baseSalary * 0.15 + teamSize * 5000`
3. Класс `Developer extends Employee`:
    - Дополнительное поле: `language` (String)
    - Бонус = `baseSalary * 0.12`
4. Класс `Intern extends Employee`:
    - Бонус = фиксированные `10000`

```java
public class EmployeeBonus {
    public static void main(String[] args) {
        Employee[] team = {
            new Manager("Ольга", 120000, 5),
            new Developer("Андрей", 95000, "Java"),
            new Developer("Мария", 100000, "Python"),
            new Intern("Стажёр Петя", 30000)
        };

        System.out.println("=== Расчёт бонусов ===");
        double totalBudget = 0;
        for (Employee e : team) {
            System.out.println(e);
            totalBudget += e.totalCompensation();
        }
        System.out.printf("\nОбщий бюджет: %.0f руб.%n", totalBudget);
    }
}
```

Ожидаемый вывод:

```
=== Расчёт бонусов ===
Ольга | Оклад: 120000.0 | Бонус: 43000.0 | Итого: 163000.0
Андрей | Оклад: 95000.0 | Бонус: 11400.0 | Итого: 106400.0
Мария | Оклад: 100000.0 | Бонус: 12000.0 | Итого: 112000.0
Стажёр Петя | Оклад: 30000.0 | Бонус: 10000.0 | Итого: 40000.0

Общий бюджет: 421400 руб.
```

Задание 2.2: Sealed-интерфейс — система оплаты

Спроектируйте систему обработки платежей с использованием sealed-интерфейса. Напишите весь код с нуля.

Требования:

1. `sealed interface PaymentMethod permits CreditCard, BankTransfer, CryptoWallet` с методами:
    - `String process(double amount)` — возвращает строку-описание выполненной операции
    - `double fee(double amount)` — возвращает комиссию
2. `record CreditCard(String cardNumber, String holder)` :
    - `process()` → `"Оплата картой *XXXX: Z руб."` (последние 4 цифры номера)
    - Комиссия = 2% от суммы
3. `record BankTransfer(String bankName, String iban)` :
    - `process()` → `"Перевод через БАНК: Z руб."`
    - Комиссия = фиксированные 50 руб.
4. `record CryptoWallet(String address, String currency)` :
    - `process()` → `"Криптоплатёж (ВАЛЮТА): Z руб."`
    - Комиссия = 1.5% от суммы
5. Класс `PaymentProcessor` со статическим методом `describe(PaymentMethod pm)` , который использует switch с паттерн-матчингом (Java 21+) для вывода подробного описания.

```java
public class PaymentDemo {
    public static void main(String[] args) {
        PaymentMethod[] payments = {
            new CreditCard("4111222233334444", "Иван Петров"),
            new BankTransfer("Сбербанк", "RU1234567890"),
            new CryptoWallet("0xABC123", "BTC")
        };

        double amount = 10000;
        for (PaymentMethod pm : payments) {
            System.out.println(pm.process(amount));
            System.out.printf("  Комиссия: %.2f руб.%n", pm.fee(amount));
            PaymentProcessor.describe(pm);
            System.out.println();
        }
    }
}
```

Задание 2.3: Интерфейсы — default, static и private методы

Напишите интерфейс `Loggable` и два класса, реализующих его.

Требования к интерфейсу `Loggable` :

1. Абстрактный метод `String getComponentName()` — возвращает имя компонента
2. `default` метод `void log(String message)` — выводит сообщение в формате `"[ВРЕМЯ] [ИМЯ_КОМПОНЕНТА] сообщение"` . Для форматирования времени используйте приватный метод `formatTimestamp()`
3. `default` метод `void logError(String message)` — аналогично, но с префиксом `"ОШИБКА: "` перед сообщением
4. `private` метод `String formatTimestamp()` — возвращает текущее время в формате `"HH:mm:ss"` (используйте `java.time.LocalTime.now()` и `java.time.format.DateTimeFormatter`)
5. `static` метод `String getLogLevel()` — возвращает `"INFO"`

Напишите два класса:

1. `DatabaseService implements Loggable` — с методом `connect(String url)` , который логирует подключение
2. `AuthService implements Loggable` — с методом `login(String username, boolean success)` , который логирует результат входа (при неудаче — через `logError`)

```java
public class LoggableDemo {
    public static void main(String[] args) {
        DatabaseService db = new DatabaseService();
        AuthService auth = new AuthService();

        System.out.println("Уровень логирования: " + Loggable.getLogLevel());
        System.out.println();

        db.connect("jdbc:postgresql://localhost/mydb");
        System.out.println();

        auth.login("admin", true);
        auth.login("hacker", false);
    }
}
```

Примерный вывод (время будет отличаться):

```
Уровень логирования: INFO
[14:30:15] [DatabaseService] Подключение к jdbc:postgresql://localhost/mydb
[14:30:15] [DatabaseService] Подключение установлено

[14:30:15] [AuthService] Вход пользователя: admin — успешно
[14:30:15] [AuthService] ОШИБКА: Вход пользователя: hacker — отказано
```

Задание 2.4: Абстрактный класс vs Интерфейс

Определите, что лучше использовать в каждом случае — абстрактный класс или интерфейс. Обоснуйте ответ, опираясь на конкретные критерии (наличие состояния, множественная реализация, отношение "является" vs "умеет").

Ситуация	Ваш выбор	Обоснование
Все животные имеют имя и возраст, но звучат по-разному		
Некоторые объекты (документ, изображение, настройки) можно сериализовать		
Все транспортные средства имеют скорость и топливо, но двигаются по-разному		
Робот, посудомоечная машина и телефон — все умеют подключаться к Wi-Fi		
Круг, квадрат и треугольник — все фигуры с площадью и периметром, у всех есть цвет		
Класс должен одновременно уметь летать, плавать и ездить		

Часть 3: Массивы

Задание 3.1: Операции с матрицами

Напишите класс `MatrixOperations` с нуля. Реализуйте статические методы для работы с двумерными массивами.

Требования:

1. `static void print(int[][] matrix)` — выводит матрицу в отформатированном виде (каждое число шириной 4 символа)
2. `static int[][] transpose(int[][] matrix)` — возвращает транспонированную матрицу (строки становятся столбцами)
3. `static int[][] multiply(int[][] a, int[][] b)` — умножение матриц. Если размеры несовместимы — выведите ошибку и верните `null`
4. `static int diagonalSum(int[][] matrix)` — сумма элементов главной диагонали

Напоминание: при умножении матриц `C[i][j] = сумма(A[i][k] * B[k][j])` для всех `k` .

```java
public class MatrixOperations {

    // Напишите все методы самостоятельно

    public static void main(String[] args) {
        int[][] a = {
            {1, 2, 3},
            {4, 5, 6}
        };

        int[][] b = {
            {7,  8},
            {9,  10},
            {11, 12}
        };

        System.out.println("Матрица A (2x3):");
        print(a);

        System.out.println("\nТранспонированная A (3x2):");
        print(transpose(a));

        System.out.println("\nМатрица B (3x2):");
        print(b);

        int[][] c = multiply(a, b);
        System.out.println("\nA * B (2x2):");
        print(c);

        System.out.println("\nСумма диагонали A*B: " + diagonalSum(c));
    }
}
```

Ожидаемый вывод:

```
Матрица A (2x3):
   1   2   3
   4   5   6

Транспонированная A (3x2):
   1   4
   2   5
   3   6

Матрица B (3x2):
   7   8
   9  10
  11  12
```

```
A * B (2x2):
  58  64
 139 154

Сумма диагонали A*B: 212
```

Задание 3.2: Зубчатый массив — журнал оценок

Напишите программу `GradeJournal.java`, которая хранит оценки студентов в зубчатом массиве (у каждого студента разное количество оценок) и выполняет анализ.

Требования:

1. Создайте массив имён: `{"Алиса", "Борис", "Вера", "Глеб"}`
2. Создайте зубчатый массив `int[][]` с оценками:
   - Алиса: 5, 4, 5, 5, 3
   - Борис: 3, 3, 4
   - Вера: 5, 5, 5, 5, 5, 4
   - Глеб: 4, 3, 4, 5
3. Напишите метод `double average(int[] grades)` — средний балл
4. Напишите метод `int max(int[] grades)` — максимальная оценка
5. Напишите метод `int min(int[] grades)` — минимальная оценка
6. В `main` выведите для каждого студента: имя, количество оценок, средний балл, мин и макс
7. Найдите и выведите имя студента с наивысшим средним баллом

Ожидаемый вывод:

```
=== Журнал оценок ===
Алиса   | Оценок: 5 | Средний: 4.40 | Мин: 3 | Макс: 5
Борис   | Оценок: 3 | Средний: 3.33 | Мин: 3 | Макс: 4
Вера    | Оценок: 6 | Средний: 4.83 | Мин: 4 | Макс: 5
Глеб    | Оценок: 4 | Средний: 4.00 | Мин: 3 | Макс: 5

Лучший студент: Вера (средний балл: 4.83)
```

Часть 4: Строки и StringBuilder

Задание 4.1: Анализатор текста

Напишите класс `TextAnalyzer` с нуля. Класс принимает текст в конструкторе и предоставляет методы анализа.

Требования:

1. Конструктор `TextAnalyzer(String text)`
2. `int wordCount()` — количество слов (разделённых пробелами)
3. `String longestWord()` — самое длинное слово
4. `String reverseWords()` — текст с обратным порядком слов (не букв!). Например: `"Привет мир Java"` → `"Java мир Привет"`
5. `int countOccurrences(String target)` — сколько раз подстрока встречается в тексте (без учёта регистра)
6. `boolean isPalindrome()` — является ли текст палиндромом (игнорируя регистр, пробелы и знаки препинания). Подсказка: оставьте только буквы через `replaceAll("[^a-zA-Za-яA-ЯёЁ]", "")`

```java
public class TextAnalyzer {

    // Напишите весь класс самостоятельно

    public static void main(String[] args) {
        TextAnalyzer ta = new TextAnalyzer("Java Programming is Fun and Java is Powerful");

        System.out.println("Текст: " + ta);
        System.out.println("Слов: " + ta.wordCount());
        System.out.println("Самое длинное слово: " + ta.longestWord());
        System.out.println("Слова наоборот: " + ta.reverseWords());
        System.out.println("'Java' встречается: " + ta.countOccurrences("java") + " раз(а)");
        System.out.println("'is' встречается: " + ta.countOccurrences("is") + " раз(а)");
        System.out.println("Палиндром: " + ta.isPalindrome());

        System.out.println();

        TextAnalyzer palindrome = new TextAnalyzer("А роза упала на лапу Азора");
        System.out.println("Текст: " + palindrome);
        System.out.println("Палиндром: " + palindrome.isPalindrome());
    }
}
```

Ожидаемый вывод:

```
Текст: Java Programming is Fun and Java is Powerful
Слов: 8
Самое длинное слово: Programming
Слова наоборот: Powerful is Java and Fun is Programming Java
```

```
'Java' встречается: 2 раз(а)
'is' встречается: 2 раз(а)
Палиндром: false

Текст: А роза упала на лапу Азора
Палиндром: true
```

Задание 4.2: String Pool — исследование

Напишите программу `StringPoolLab.java` , которая исследует поведение String Pool. Вы должны самостоятельно создать все примеры и предсказать результаты **до** запуска.

Требования:

1. Создайте строки 6 разными способами:
    - `s1` — строковый литерал `"Hello"`
    - `s2` — ещё один литерал `"Hello"`
    - `s3` — через `new String("Hello")`
    - `s4` — через `new String("Hello")`
    - `s5` — через `s3.intern()`
    - `s6` — результат конкатенации: `"Hel" + "lo"` (литералы)
    - `s7` — результат конкатенации с переменной: `String half = "Hel"; String s7 = half + "lo";`
2. Для каждой пары выведите результат `==` и `.equals()` :
    - `s1` и `s2`
    - `s1` и `s3`
    - `s3` и `s4`
    - `s1` и `s5`
    - `s1` и `s6`
    - `s1` и `s7`
3. **Перед каждым сравнением** напишите в комментарии ваш прогноз и объяснение.
4. В конце программы используя `StringBuilder` соберите строку `"Hello"` по буквам и сравните её с `s1` через `==` и `.equals()` .

Часть 5: Records, Enums, EnumSet и EnumMap

Задание 5.1: Система оценок

Напишите систему оценок студентов с нуля. Используйте `record` , `enum` , `EnumMap` и `EnumSet` .

Требования:

1. `enum Grade` с константами `A` , `B` , `C` , `D` , `F` :
    - Поле `String description` (Отлично, Хорошо, Удовлетворительно, Неудовлетворительно, Провал)
    - Поле `double gpaValue` (4.0, 3.0, 2.0, 1.0, 0.0)
    - Конструктор, геттеры
    - Метод `boolean isPassing()` — `true` если не `F` и не `D`
    - Метод `static Grade fromScore(int score)` — преобразует числовую оценку (0–100) в Grade: A ≥ 90, B ≥ 80, C ≥ 70, D ≥ 60, иначе F
2. `record Student(String name, int id)` с компактным конструктором, который:
    - Проверяет, что `name` не `null` и не пустое
    - Проверяет, что `id > 0`
    - Выбрасывает `IllegalArgumentException` при нарушении
3. В `main` :
    - Создайте 6–7 студентов и присвойте им числовые оценки (используйте `Grade.fromScore()`)
    - Используйте `EnumMap<Grade, List<Student>>` для группировки студентов по оценкам
    - Используйте `EnumSet` для получения множества "проходных" оценок
    - Выведите сводку: для каждой оценки — список студентов и их количество
    - Подсчитайте средний GPA всех студентов

```java
import java.util.*;

public class GradeSystem {

    // Напишите enum Grade, record Student и main самостоятельно

    public static void main(String[] args) {
        // Создайте студентов и оценки
        // Сгруппируйте через EnumMap
        // Выведите сводку
    }
}
```

Задание 5.2: Record с бизнес-логикой и Enum с абстрактным методом

Часть A: Напишите `record Temperature(double value, Unit unit)` :

- `enum Unit { CELSIUS, FAHRENHEIT, KELVIN }`
- Компактный конструктор: проверьте, что значение в Кельвинах не ниже 0 (абсолютный ноль). Для этого во всех единицах переведите в Кельвины и проверьте.
- Метод `Temperature convertTo(Unit targetUnit)` — конвертирует температуру
- Переопределённый `toString()` — формат: `"36.6 °C"` или `"97.88 °F"` или `"309.75 K"`

Часть B: Напишите `enum MathOperation` :

- Константы: `ADD` , `SUBTRACT` , `MULTIPLY` , `DIVIDE`
- У каждой константы — абстрактный метод `double apply(double a, double b)` , реализованный индивидуально
- У `DIVIDE` — проверка деления на ноль (выбросить `ArithmeticException`)

```java
public class RecordEnumDemo {
    public static void main(String[] args) {
        Temperature body = new Temperature(36.6, Temperature.Unit.CELSIUS);
        System.out.println(body);
        System.out.println(body.convertTo(Temperature.Unit.FAHRENHEIT));
        System.out.println(body.convertTo(Temperature.Unit.KELVIN));

        System.out.println();

        double a = 10, b = 3;
        for (MathOperation op : MathOperation.values()) {
            System.out.printf("%s(%g, %g) = %g%n", op.name(), a, b, op.apply(a, b));
        }
    }
}
```

Часть 6: Аннотации

Задание 6.1: Собственная аннотация + обработка через Reflection

Это задание состоит из двух частей. В первой части заполните пропуски (синтаксис аннотаций). Во второй части напишите дополнительную аннотацию с нуля.

Часть A: Заполните пропуски

Дополните объявление аннотации `@TestInfo` :

```java
import java.lang.annotation.*;
import java.lang.reflect.Method;

@Retention(____)      // доступна во время выполнения через Reflection
@Target(____)         // применяется к методам
@interface TestInfo {
    ____;             // String author()
    ____;             // String date()
    ____;             // String description() default ""
    ____;             // int priority() default 5
}
```

Часть B: Напишите полностью с нуля

Создайте файл `ValidationFramework.java` — мини-фреймворк валидации с аннотациями и Reflection.

Требования:

1. Аннотация `@NotEmpty` — применяется к полям типа `String` , RetentionPolicy = RUNTIME. Параметр: `String message() default "Поле не может быть пустым"`
2. Аннотация `@Range` — применяется к полям типа `int` , RetentionPolicy = RUNTIME. Параметры: `int min()` , `int max()` , `String message() default "Значение вне допустимого диапазона"`
3. Класс `RegistrationForm` с полями:
   - `@NotEmpty(message = "Имя обязательно") String name`
   - `@NotEmpty String email`
   - `@Range(min = 18, max = 120, message = "Возраст должен быть от 18 до 120") int age`
4. Класс `Validator` со статическим методом `List<String> validate(Object obj)` , который через Reflection:
   - Проходит по всем полям объекта
   - Для `@NotEmpty` — проверяет, что строка не `null` и не пустая
   - Для `@Range` — проверяет, что число в указанном диапазоне
   - Возвращает список сообщений об ошибках (пустой список = валидация пройдена)

```java
public class ValidationFramework {
    public static void main(String[] args) {
        RegistrationForm valid = new RegistrationForm("Иван", "ivan@mail.ru", 25);
        RegistrationForm invalid = new RegistrationForm("", null, 15);

        System.out.println("=== Валидация корректной формы ===");
```

```java
        List<String> errors1 = Validator.validate(valid);
        System.out.println(errors1.isEmpty() ? "Все поля валидны!" : errors1);

        System.out.println("\n=== Валидация некорректной формы ===");
        List<String> errors2 = Validator.validate(invalid);
        errors2.forEach(e -> System.out.println("  - " + e));
    }
}
```

Ожидаемый вывод:

```
=== Валидация корректной формы ===
Все поля валидны!

=== Валидация некорректной формы ===
  - Имя обязательно
  - Поле не может быть пустым
  - Возраст должен быть от 18 до 120
```

Часть 7: Анонимные классы, локальные классы, лямбды и ссылки на методы

Задание 7.1: Эволюция кода — от анонимного класса к ссылке на метод

Вам дан код, использующий анонимные классы. Выполните три этапа рефакторинга.

Этап 1: Перепишите этот код — замените каждый анонимный класс на **лямбда-выражение**. Сохраните в файле `RefactorStep1.java` .

Этап 2: Там, где это возможно, замените лямбды на **ссылки на методы**. Сохраните в файле `RefactorStep2.java` .

Этап 3: В комментариях объясните, какие лямбды **нельзя** заменить на ссылки на методы и почему.

```java
import java.util.*;
import java.util.function.*;

public class RefactorOriginal {
    public static void main(String[] args) {
        List<String> cities = Arrays.asList("Москва", "Берлин", "Токио", "Нью-Йорк", "Париж");

        // 1. Сортировка по длине
        cities.sort(new Comparator<String>() {
            @Override
            public int compare(String a, String b) {
                return Integer.compare(a.length(), b.length());
            }
        });

        // 2. Вывод каждого элемента
        cities.forEach(new Consumer<String>() {
            @Override
            public void accept(String city) {
                System.out.println(city);
            }
        });

        // 3. Преобразование в верхний регистр
        Function<String, String> toUpper = new Function<String, String>() {
            @Override
            public String apply(String s) {
                return s.toUpperCase();
            }
        };

        // 4. Проверка длины > 5
        Predicate<String> isLong = new Predicate<String>() {
            @Override
            public boolean test(String s) {
                return s.length() > 5;
            }
        };

        // 5. Формирование строки с восклицательным знаком
        Function<String, String> exclaim = new Function<String, String>() {
            @Override
            public String apply(String s) {
                return s + "!";
            }
        };

        // 6. Создание нового списка
        Supplier<List<String>> listFactory = new Supplier<List<String>>() {
            @Override
```

```
            public List<String> get() {
                return new ArrayList<>();
            }
        };

        // Использование
        List<String> result = listFactory.get();
        for (String city : cities) {
            if (isLong.test(city)) {
                result.add(toUpper.apply(city));
            }
        }
        System.out.println("Длинные города: " + result);
    }
}
```

Задание 7.2: Конвейер обработки данных (Stream API)

Напишите программу `OrderAnalytics.java` с нуля. Используйте Stream API, лямбды и ссылки на методы для обработки данных.

Дано:

```java
record Order(String customer, String product, double price, int quantity, String category) {
    double total() { return price * quantity; }
}
```

Создайте список заказов (10+) и выполните следующие операции через Stream API:

1. Найдите все заказы дороже 5000 руб. (по `total()`) и выведите их
2. Получите список уникальных имён клиентов (`List<String>`), отсортированный по алфавиту
3. Вычислите общую выручку (сумма всех `total()`)
4. Найдите самый дорогой заказ (по `total()`)
5. Подсчитайте количество заказов в каждой категории (`Map<String, Long>`)
6. Вычислите среднюю стоимость заказа по каждому клиенту (`Map<String, Double>`)
7. Разделите заказы на две группы: дорогие (total > 3000) и дешёвые (`Map<Boolean, List<Order>>`)
8. Получите список названий товаров дороже 1000 руб., без дубликатов, в верхнем регистре

Используйте ссылки на методы вместо лямбд везде, где это возможно (например, `Order::total` , `System.out::println`).

Задание 7.3: Композиция функций и локальный класс

Напишите программу `TextPipeline.java` .

Часть A — Композиция функций:

Создайте набор `Function<String, String>` и скомпонуйте их в конвейер:

1. `trim` — удаляет пробелы по краям
2. `lower` — переводит в нижний регистр
3. `removeExtraSpaces` — заменяет множественные пробелы на один (через `replaceAll("\\s+", " ")`)
4. `capitalize` — делает первую букву заглавной

Скомпонуйте их через `andThen()` в одну функцию `normalize` и примените к нескольким строкам.

Часть B — Локальный класс:

Внутри метода `main` объявите локальный класс `WordCounter` , который:

- Принимает строку в конструкторе
- Имеет метод `Map<String, Integer> count()` — возвращает частотный словарь слов (сколько раз каждое слово встречается)
- Имеет метод `String mostFrequent()` — возвращает самое частое слово

Используйте `WordCounter` для анализа нормализованного текста.

Часть 8: Интеграционное задание

Задание 8.1: Система управления библиотекой

Создайте файл `LibrarySystem.java` — полноценную мини-систему, объединяющую все темы лекции.

Требования:

1. **`enum Genre`** — жанры книг: `FICTION` , `SCIENCE` , `HISTORY` , `PROGRAMMING` , `ART` . Каждый жанр имеет поле `String russianName` . Метод `static Genre fromString(String name)` — находит жанр по русскому названию.

2. **`record Book(String title, String author, int year, Genre genre, double price)`** — с компактным конструктором:

    - `title` и `author` не пустые
    - `year` от 1450 до текущего года
    - `price >= 0`

3. **`sealed interface LibraryItem permits PhysicalBook, EBook`** — с методом `String getInfo()` :

   - `record PhysicalBook(Book book, String shelf) implements LibraryItem`
   - `record EBook(Book book, String format, double sizeMB) implements LibraryItem`

4. `interface Searchable` — с default-методом `boolean matches(String query)` и static-методом `static <T extends Searchable> List<T> search(List<T> items, String query)`

5. **Класс `Library`:**
   - Хранит список `LibraryItem`
   - Метод `void add(LibraryItem item)`
   - Метод `void printCatalog()` — выводит все книги, используя паттерн-матчинг switch для `LibraryItem`
   - Метод `Map<Genre, List<LibraryItem>> groupByGenre()` — группирует через `EnumMap` и Stream API
   - Метод `double totalValue()` — общая стоимость через stream + reduce
   - Метод `Optional<LibraryItem> mostExpensive()` — самая дорогая книга
   - Метод `List<String> authorsByGenre(Genre genre)` — уникальные авторы жанра, отсортированные по алфавиту

6. **В `main`:** создайте библиотеку с 8+ книгами (физические и электронные), продемонстрируйте все методы.

Часть 9: Эксперименты в jshell

Задание 9.1: Sealed-классы

Выполните в jshell (требуется Java 17+):

```
jshell> sealed interface Shape permits Circle, Square {}
jshell> record Circle(double r) implements Shape {}
jshell> record Square(double side) implements Shape {}
jshell> Shape s = new Circle(5)
jshell> s instanceof Circle c ? "Круг r=" + c.r() : "Не круг"
```

Вопрос: Попробуйте создать `record Triangle(double a) implements Shape {}`. Что произойдёт и почему?

Задание 9.2: Цепочка лямбд

```
jshell> import java.util.function.*
jshell> Function<String, String> trim = String::trim
jshell> Function<String, String> upper = String::toUpperCase
jshell> Function<String, String> exclaim = s -> s + "!"
jshell> var pipeline1 = trim.andThen(upper).andThen(exclaim)
jshell> var pipeline2 = exclaim.compose(upper).compose(trim)
jshell> pipeline1.apply("  hello world  ")
jshell> pipeline2.apply("  hello world  ")
```

Вопрос: Дают ли `andThen()` и `compose()` одинаковый результат в данном случае? В каком случае результаты будут различаться?

Задание 9.3: Сравнение EnumSet и HashSet

```
jshell> enum Color { RED, GREEN, BLUE, YELLOW, CYAN, MAGENTA, WHITE, BLACK }
jshell> var enumSet = java.util.EnumSet.of(Color.RED, Color.GREEN, Color.BLUE)
jshell> var hashSet = new java.util.HashSet<>(java.util.Set.of(Color.RED, Color.GREEN, Color.BLUE))
jshell> enumSet.contains(Color.RED)
jshell> hashSet.contains(Color.RED)
jshell> enumSet.getClass().getSimpleName()
jshell> hashSet.getClass().getSimpleName()
```

Вопрос: Внутренний класс `EnumSet` называется `RegularEnumSet`. Почему? Что произойдёт, если enum будет иметь больше 64 констант?

Часть 10: Контрольные вопросы

Ответьте письменно. В вопросах, где дан код — предскажите результат **без запуска**.

1. Что выведет этот код? Объясните порядок выполнения блоков.

```java
class Demo {
    static { System.out.print("A "); }
    { System.out.print("B "); }
    Demo() { System.out.print("C "); }
    public static void main(String[] args) {
        System.out.print("D ");
        new Demo();
        System.out.print("E ");
        new Demo();
    }
}
```

2. Класс `Child` находится в пакете `b`, а класс `Parent` — в пакете `a`. `Child extends Parent`. Какие члены `Parent` доступны в

`Child` ? А какие доступны в другом классе пакета `b` , который **не** наследует `Parent` ?

3. Можно ли создать sealed-класс, у которого все наследники — record? Какое преимущество это даёт при использовании в switch?

4. Что выведет этот код? Объясните, что такое String Pool и как работает `intern()` .

```java
String a = "Hello";
String b = new String("Hello");
String c = b.intern();
System.out.println(a == b);
System.out.println(a == c);
System.out.println(b == c);
```

5. Почему этот код не компилируется? Как исправить?

```java
interface A { default void hello() { System.out.println("A"); } }
interface B { default void hello() { System.out.println("B"); } }
class C implements A, B { }
```

6. В чём разница между `StringBuilder` и `StringBuffer` ? Приведите конкретный сценарий, когда нужен именно `StringBuffer` .

7. Что выведет этот код? Объясните, что происходит с переменной `x` в лямбде.

```java
int x = 10;
Runnable r = () -> System.out.println(x);
// x = 20;  // что будет, если раскомментировать?
r.run();
```

8. Напишите пример лямбда-выражения, которое **нельзя** заменить на ссылку на метод. Объясните почему.

9. Чем `EnumMap` быстрее `HashMap` для ключей-enum? Опишите внутреннюю реализацию в одном предложении.

10. Дан `record Point(int x, int y) {}` . Что сгенерирует компилятор автоматически? Можно ли добавить в record изменяемое поле? Можно ли наследоваться от record?

Результаты занятия

К концу занятия вы должны сдать:

1. Файлы `.java` со всеми выполненными заданиями (Части 1–8)
2. Ответы на контрольные вопросы (Часть 10)
3. Заметки о результатах экспериментов в jshell (Часть 9)